MDMC
Master in Data Management and Curation

AREA SCIENCE PARK

SISSA

Python II: Exceptions, warnings and testing

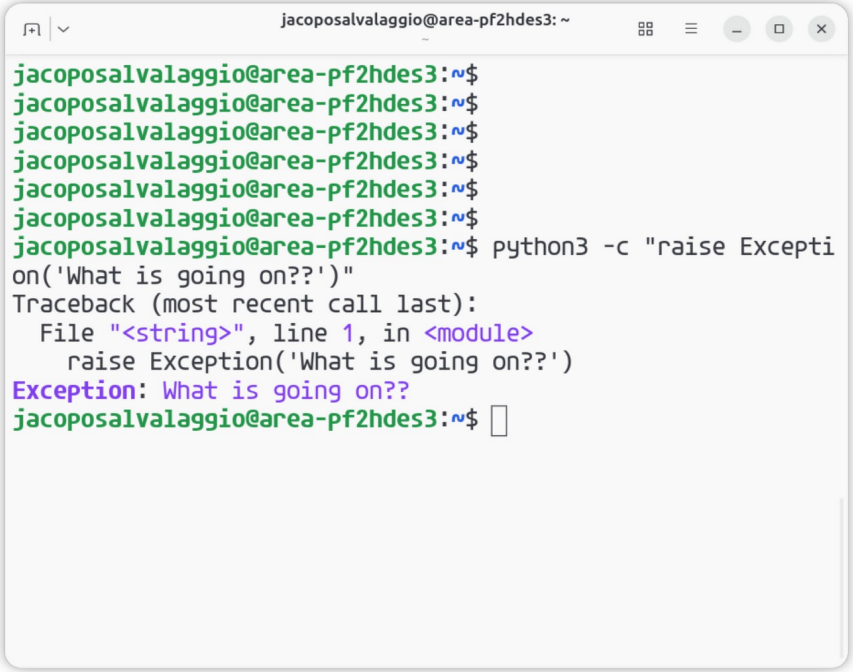
Jacopo Salvalaggio



Image credit: freeiconspng.com

CC BY

Goals



Objectives of the lecture:

- Learn how to make the code **stop** for errors
- Implement **error handling** and **warnings**
- Prepare automated **unit tests**

Exceptions

Exceptions are **errors** detected **during execution** that don't need to be fatal.

They can be **handled by the user** and the flow of the code can be altered depending on the occurrence of these exceptions.

Example: an exception

```
>>> a
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    a
NameError: name 'a' is not defined
```

Some examples

Some very common exceptions are:

* `Exception` : general exception;
* `ValueError` : when the value of a variable is not accepted
* `TypeError` : when the type of a variable is not accepted
* `NameError` : when attempting to find a name that does not exist
* `RuntimeError` : non-specific error when running code

Raising an exception

You can tell Python to invoke an error with the **`raise`** statement.

You can also pass a string as an argument of the exception to have it printed when it is triggered

Example: raising

```
def some_function(positive_number):
    if positive_number < 0:
        raise ValueError("Number must be positive!")
```

Try and see

You might want to **decide what happens when an exception is raised** and not have it just stop the code.

You can do that with the **`try - except`** statement:

Example: trying

```
try:
    x = some_function(some_value)
except:
    x = 0
    print("Oops, something went wrong...")
```

Accessing the exception

When writing the except clause, we can **access** the original exception.

Example: recovering the exception

```
try:
    x = some_function(some_value)
except Exception as e:
    x = 0
    print("Oops, something went wrong...")
    print(f"An exception was raised with the following
message(s): {e.args}")
```

Handling many exception types

It is possible to **handle** the error **differently depending** on the **type** of exception raised.

Exception is the parent class of most errors so it works with anything*.

Example: a more refined example

```
try:
    x = some_function(some_value)
except ValueError:
    x = 0
    print("Bad value passed!")
except NameError as name_error:
    print("Unknown variable name!")
    raise name_error
except Exception as exception:
    print("An error occurred!")
    raise exception
```

`else` and `finally`

You can add two other clauses to the try statement:

* **`else`** will execute all the code inside **if no error** is raised
* **`finally`** will **always run** at the end of the statement, whether errors have been raised or not

Example: extending the try statement

```
try: ...
except: ...
else:
    print("No exceptions raised")
finally:
    some_necessary_code()
```

Warnings

Warnings tell the user they are doing something that they should not, but they are **not as serious as exceptions**.

They do not interrupt the code and are not handled with `try`

Example: a warning

```
from warnings import warn
def some_func(better_if_positive):
    if better_if_positive < 0:
        warn("This number should be positive...")
```

Unit tests

You can **test** that **your code** does what you expect it to do with unit tests.

There are many libraries that do it, the standard builtin for Python is **`unittest`**

The fundamental tests you can do are **asserting** that **two values are equal**, asserting whether **a condition is true** or false and asserting that **an error is raised** in some context.

An example

Example: a simple unit test

```
from unittest import TestCase
import mycode
class TestCode(TestCase):
    def test_result(self):
        self.assertEqual(
            1,
            mycode.must_return_one()
        )
    def test_typecheck(self):
        with self.assertRaises(TypeError):
            x = mycode.argument_is_string(1)
```

Checking the code returns the expected value

Checking the code raises the right error

Running the test

Example: running a test

```
$ python3 -m unittest some_test.py
.F
======================================================================
FAIL: test_typecheck (some_test.TestCode.test_typecheck)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/home/jacoposalvalaggio/testing/some_test.py", line 8, in test_typecheck
    with self.assertRaises(TypeError):
         ~~~~~~~~~~~~~~~~~~^^^^^^^^^^^
AssertionError: TypeError not raised


----------------------------------------------------------------------
Ran 2 tests in 0.001s


FAILED (failures=1)
```

Does this matter?

Yes! In production code, you **must be sure that everything behaves as expected**. When something happens that you did not account for, the end user gets a bug!

Unit tests provide an automated way of checking that nothing goes bad.

(For an interesting point of view see: test-driven development)

Unit tests humor

A software tester walks into a bar. Runs into a bar. Crawls into a bar. Dances into a bar. Flies into a bar. Jumps into a bar.

And orders:
- a beer.
- 2 beers.
- 0 beers.
- 99999999 beers.
- a lizard in a beer glass.
- -1 beer.
- "qwertyuiop" beers.

Testing complete.

A real customer walks into the bar and asks where the bathroom is.
The bar goes up in flames.

AREA SCIENCE PARK

SISSA

OGS Istituto Nazionale di Oceanografia e di Geofisica Sperimentale

ARCTUR

REGIONE AUTONOMA FRIULI VENEZIA GIULIA

E-ARGO



Image credit: Abel+Imray

Thank you for your attention :)

Image credits for background rainbow gradient: Image Colorful Rainbow Pattern by sgagnonou812 at Wallpapers.com